



A Practical Guide to Effective Requirements Development

Research indicates that nearly 50% of all software project defects originate in the requirements gathering process and that 60% to 80% of project failures can be attributed directly to poor requirements gathering. In the United States alone, this represents a \$60 billion annual problem involving cancelled or failed projects. This E-Guide examines requirements gathering, specifically some of the models that can be used when gathering requirements, an overview of use cases and five traps to avoid when using them, planning requirements for multiple product releases, as well as guidance on how to effectively avoid the common pitfalls and costs directly attributed to poor requirements gathering.

Sponsored By:

RAVENFLOW
requirements
development
ELICIT ► ANALYZE ► SPECIFY ► VALIDATE



A Practical Guide to Effective Requirements Development

Table of Contents

[Using models to define, understand users' needs By Ellen Gottesdiener](#)

Ellen Gottesdiener, Principal Consultant, EBG Consulting, helps teams to collaboratively explore requirements, shape their development processes, and plan and review their work. Her book, *Requirements by Collaboration: Workshops for Defining Needs* (Addison-Wesley, 2002), describes how to use multiple models to elicit requirements in collaborative workshops. Ellen's most recent book, *The Software Requirements Memory Jogger: A Pocket Guide to Help Software and Business Teams Develop and Manage Requirements* (GOAL/QPC, 2005), describes essentials for requirements development and management. Both are available on Amazon.com and via other quality booksellers.

[Ambiguous requirements lead to confusion, extra work By Karl E. Wiegers](#)

[Five use case traps to avoid By Karl E. Wiegers](#)

[Planning requirements for multiple product releases By Karl E. Wiegers](#)

Karl E. Wiegers is Principal Consultant with Process Impact in Portland, Ore. He is also the author of *More About Software Requirements: Thorny Issues and Practical Advice*, *Software Requirements, 2nd Edition*; *Peer Reviews in Software: A Practical Guide*; and *Creating a Software Engineering Culture*.

[White Paper from Ravenflow: How to Detect Requirements Errors By Adam Frankl and Tom King](#)

Using models to define, understand users' needs

By Ellen Gottesdiener for SearchSoftwareQuality.com

To deliver the right product, you need to articulate your requirements early on, before the cost of fixing requirements errors kills your development budget, generates customer ill will and jeopardizes your business. If you don't obtain user requirements through efficient means, you won't deliver the right product.

The problem is that requirements are not just sitting around waiting to be written down. It takes a lot of effort and skill to elicit, analyze and verify them.

The most popular, traditional way to understand user needs is to write a set of textual requirements ("the system shall..."). But writing these statements is not enough. Textual requirements have their place when you need formal specifications, but they rarely communicate user needs effectively.

Understanding user needs is both an art and a science — a combination of discovery, investigation and decision making. Successful projects engage users early and then explore and reach closure on their requirements by using *analysis models* — representations of user requirements.

What models tell you about your project

Models are approximations of reality. For software projects, analysis models depict user needs represented by text (such as tables, lists or matrices), diagrams or a combination of the two types. Model types include context diagrams, scenarios, data models, event-response tables, state diagrams and business rules.

Figure 1 shows how user requirements models answer the questions *Who? What? When? Why?* and *How?* Each model represents information at different levels of abstraction. For example, a state diagram represents information at a high level of abstraction, whereas detailed textual requirements represent a low level of abstraction. By first understanding the forest (a state diagram) before examining the trees (textual requirements), you can discover requirements defects that aren't evident when you review textual requirements alone.

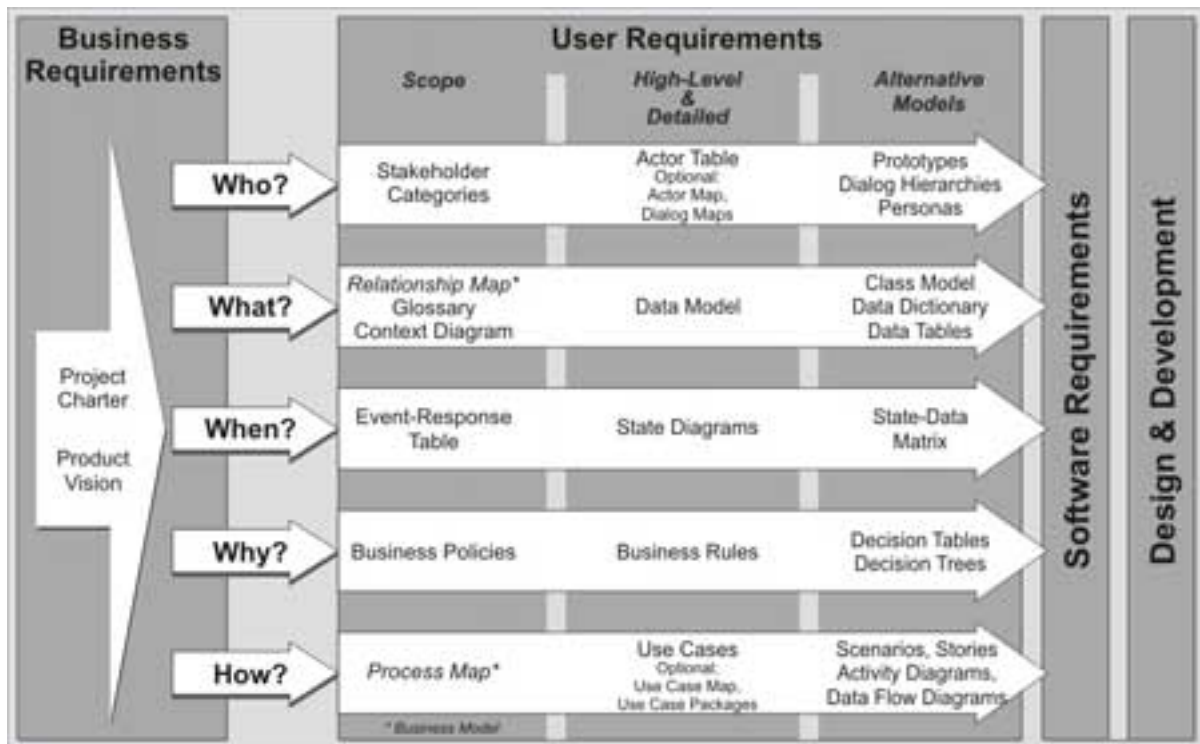


Figure 1: Requirements Roadmap
Copyright EBG Consulting, 2007

In my experience, analysis models created in collaborative workshops involving business and technical stakeholders are one of the quickest ways to articulate requirements, reveal missing and conflicting requirements, and crystallize user needs.

For example, recently I worked with a project team on Project TES. Before I came on board, the business owner and technical staff were passing many documents back and forth but making little progress in developing the TES requirements. It wasn't until we gathered together and started modeling requirements that the scope of the project became clear.

We used a combination of models — a context diagram and an event-response table — to clarify project scope. **Figure 2** shows the context diagram for project TES.

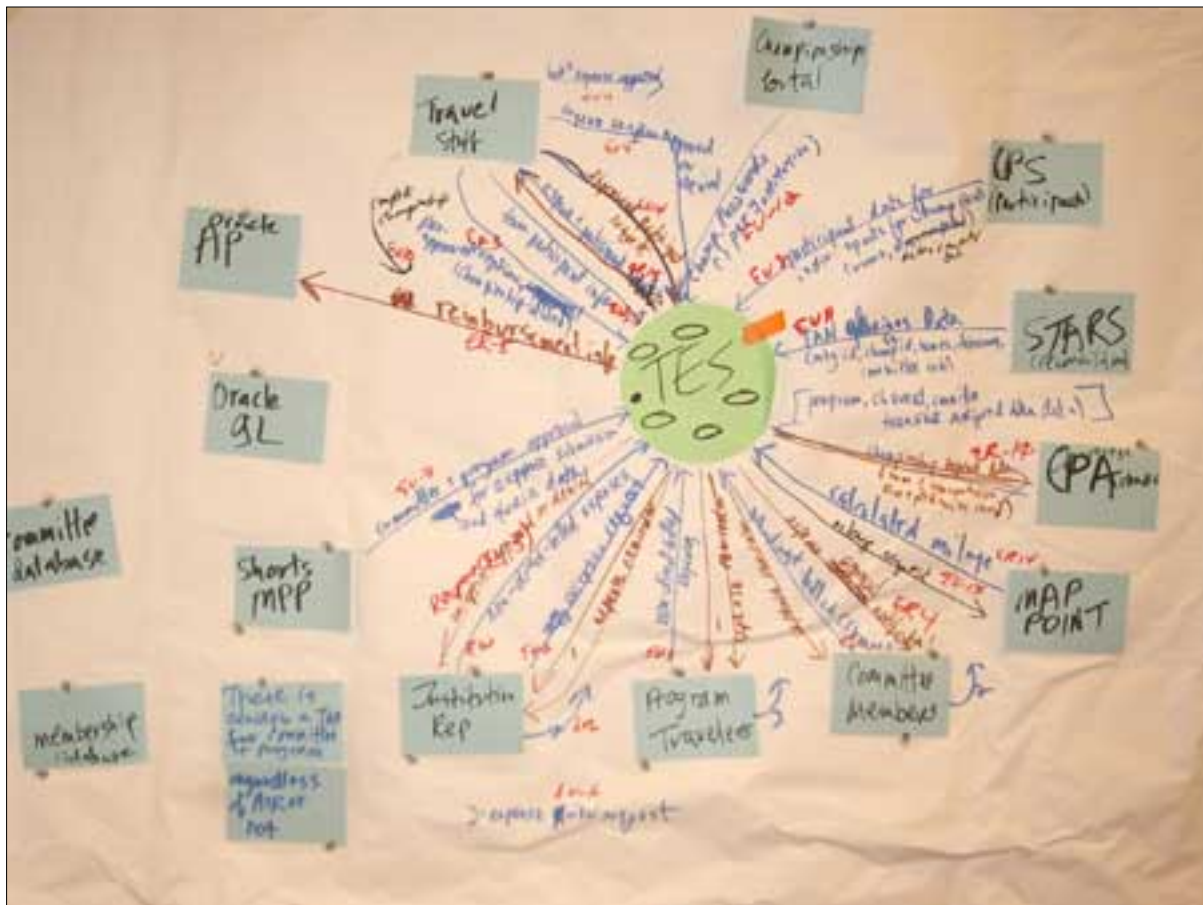


Figure 2: Context Diagram for Project TES
Source: EBG Consulting

The team created the context diagram in less than an hour. Meanwhile, a recorder captured matching events and their responses in a spreadsheet that was projected for everyone to see.

This modeling activity quickly revealed system interfaces that had not been understood by the technical staff before. It also raised questions about the source of some data — questions that only the business people could answer.

Once the team recognized the complexity of the project, it was much easier for them to discuss an approach for tackling requirements in more detail. They concluded that they needed to define the requirements in logical chunks using iterations. A bit later, the team gathered to successfully elaborate the requirements using additional models (use cases, actors, scenarios, business rules, a state diagram and a prototype).

Seeing the biggest picture

In addition to representing user requirements before software specification, requirements models are useful for understanding an entire business process before you define a software solution.

For example, consider a project I'll call Project EX. A global supply-chain division wanted to alter its business processes, which involved forecasting, allocating finished goods and invoicing.

The business program managers assumed that they knew their software requirements. But after they launched the project, it became clear that there were gaps in their understanding of the overall end-to-end supply-chain process.

Again, analysis models came to the rescue. We gathered all the business program managers in a facilitated workshop to create a series of interconnected models — a relationship map, a process map, state diagrams and scenarios. **Figure 3** shows a portion of one of the state diagrams.

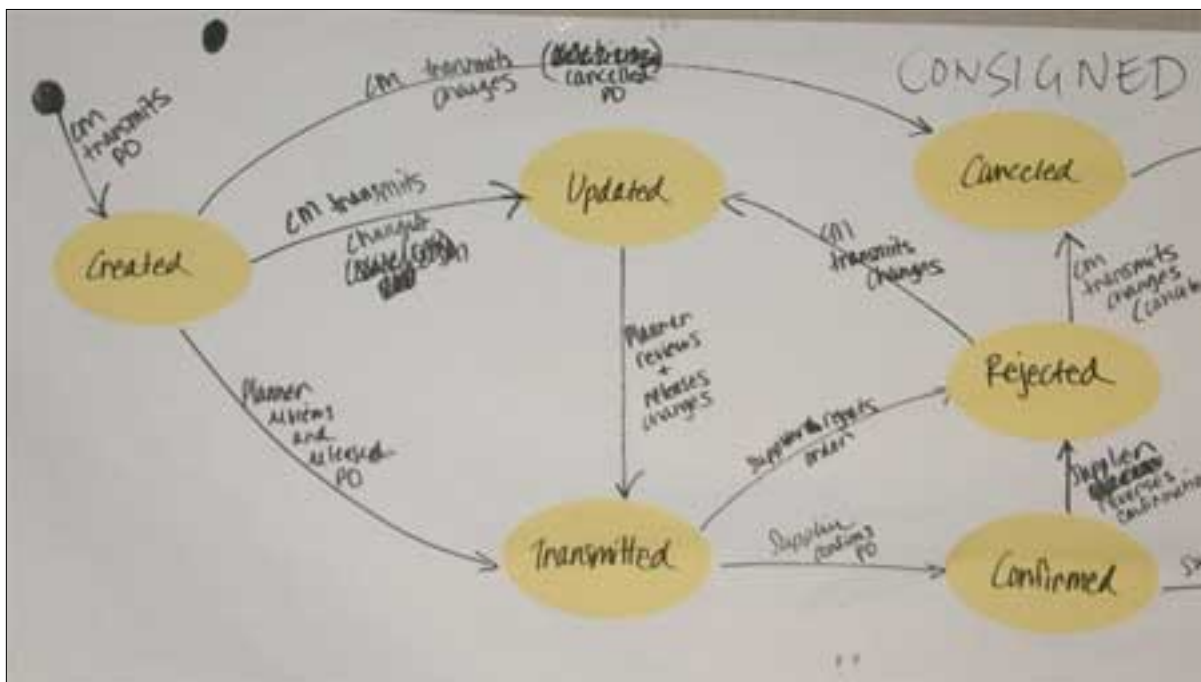


Figure 3: State Diagram for a Supply Chain (a partial *consigned purchase order*)
Source: EBG Consulting

Using the models to explore their needs enabled the business people to significantly rethink the overall business process and to better understand the dependencies inherent in the supply chain. Consequently, the business owners changed their thinking about how software would help them solve their business process needs.

The more, the merrier

Because requirements models are related, developing one model leads to deriving another. For example, when developing use cases, you can develop related models:

- Actors initiate use cases.
- Scenarios exemplify instances of use cases.
- A use case acts upon data.
- A use case is governed by business rules.
- Events trigger use cases.

The models thread together, so you can use various routes to harvest one model from another. You can develop the models quickly while at the same time verifying their completeness and correctness.

Using multiple interwoven models taps into different modes of human thinking and lets you explore different aspects of user requirements from different perspectives. Some people think more precisely with words, whereas others are better able to grasp concepts quickly by studying diagrams. Using both types of representation leverages different thinking modes and permits project stakeholders to understand requirements from more than one angle.

The bottom line

Requirements elicitation and analysis is a process of learning and discovery. The goal is to sufficiently understand your requirements so that your customers can prioritize them and allocate them to software. Models can supplement — and, in some cases, substitute — requirements specified as textual statements. Because each representation relates to another, using multiple models tends to accelerate understanding and reveal errors in requirements. Analysis modeling promotes overall software quality, helping you define the right requirements so that you can deliver the right solution.

About the author: *Ellen Gottesdiener, Principal Consultant, EBG Consulting, helps teams to collaboratively explore requirements, shape their development processes, and plan and review their work. Her book, Requirements by Collaboration: Workshops for Defining Needs (Addison-Wesley, 2002), describes how to use multiple models to elicit requirements in collaborative workshops. Ellen's most recent book, The Software Requirements Memory Jogger: A Pocket Guide to Help Software and Business Teams Develop and Manage Requirements (GOAL/QPC, 2005), describes essentials for requirements development and management. Both are available on Amazon.com and via other quality booksellers.*

You can subscribe to "Success with Requirements," EBG Consulting's free, monthly e-newsletter: www.ebgconsulting.com/newsletter.php. When you sign up, you'll receive a free .pdf article by our senior associate, Mary Gorman, on an essential requirements modeling technique. We created "Success with Requirements" to help you get the right requirements so your projects start smart and deliver the right product at the right time.

Ambiguous requirements lead to confusion, extra work

By Karl E. Wiegers, Ph.D. for SearchSoftwareQuality.com

Many software requirements suffer from ambiguity. Ambiguity means that a single reader can interpret the requirement in more than one way or that multiple readers come to different interpretations. In either case, ambiguous requirements lead to confusion, wasted effort and rework. This article describes several common types of requirements ambiguity and suggests how to avoid them.

Negative requirements

Negative, or inverse, requirements state what the system will not do. Here's an example from an actual project: "All users with three or more accounts should not be migrated." Try to rephrase negative requirements into a positive sense: "The system shall migrate only users having fewer than three accounts." When changing a negative requirement into a positive one, you often need to insert the word "only" to clarify the conditions that permit the system action to take place. Double and triple negatives are especially confusing; avoid them in all situations.

Boundary conditions

Boundaries between numerical ranges or date ranges are a common source of missing requirements. One requirement might describe what the system does if the amount of the sale is less than \$100, while a second requirement describes the behavior if the amount is more than \$100. But what happens if it's exactly \$100? That's not defined. Similarly, if two requirements are written so that the endpoint of a range appears in both requirements, the expected behavior is ambiguous. Use the words "inclusive" or "exclusive" to make it clear whether the endpoints of the range are included or not.

Synonyms and near synonyms

Use specialized terms consistently throughout the project documentation. A requirements specification is not a place to creatively vary your language in an attempt to keep the reader's interest. If your project involves several terms that have similar but not identical meanings, put those into a glossary so everyone knows where to go for the exact definitions.

Pronouns

Pronouns offer another opportunity for confusion if the antecedent for each pronoun is not absolutely clear. If you say "this" or "that," there should be no confusion in the reader's mind as to what you are referring to.

Abbreviations *i.e.* and *e.g.*

These abbreviations are often misused. The abbreviation *i.e.* means “that is;” whatever follows is a complete list of items in the stated category. The abbreviation *e.g.* means “for example,” so the items that follow are merely representatives of the complete set. These abbreviations are so often used incorrectly that I don’t trust them in requirements. Use English words instead of Latin abbreviations to avoid any confusion.

Adverbs

Adverbs are subjective and qualitative, and they inevitably result in diverse individual interpretations. Here’s an illustration from an actual specification: “Generally incurs a ‘per unit’ cost...” But this requirement did not provide any indication regarding the conditions under which we would not incur a per unit cost or what to do then. I don’t know how to determine whether we’ve satisfied a requirement that says “Provide a reasonably predictable end user experience.” I guess it doesn’t have to be completely predictable.

A/B

With rare exceptions, such as “input/output,” avoid using the A/B writing construct, as in “feature/function.” This can be interpreted in many ways. Is a feature the same as a function? Are we referring to features and functions? Features or functions? Features divided by functions? Sometimes this A/B notation means the author isn’t really sure exactly what to say so he’s leaving it up to all readers to interpret as they wish. This is an invitation to confusion.

It’s important for requirements to be precise because our objective is clear and effective communication. Watch out for natural language expressions that can be read in a variety of ways.

Five use case traps to avoid

By Karl E. Wiegers, Ph.D. for SearchSoftwareQuality.com

Use cases have become a popular requirements development technique. Focusing on users and their goals, rather than on product features, improves the chances of developing a software package that truly meets customer needs. However, a mystique has grown up around use cases, and many organizations struggle to use them successfully. Here are several traps to avoid as your requirements analysts begin to apply the use case technique.

Trap #1: Use cases that users don't understand

Use cases are a way to represent user requirements, which describe what the user needs to be able to do with the product. Use cases should focus on tasks a user needs to accomplish with the help of the system, so they should relate to the user's business processes. Your users should be able to read and review use cases to find possible problems, such as missing alternative flows or incorrectly handled exceptions. If users cannot relate to use cases, there's a problem. Perhaps they're written too much from a technical, rather than business, perspective.

Trap #2: Too many use cases

When analysts are generating scores or hundreds of use cases, something's wrong. This normally means that the use cases are written at too low an abstraction level. Each use case should be general enough to cover several related scenarios on a common theme. Some of these will be success scenarios, and others represent conditions in which the use case might not succeed — exceptions. If you are caught in a use case explosion, try moving up the abstraction level to group together similar use cases, treating them as alternative flows of a single, more abstract use case.

Trap #3: Overly complex use cases

The general guideline is that the number of steps in the normal flow of a use case should not exceed approximately a dozen. I once read a use case with nearly 50 steps in the normal flow. The problem was that the "normal flow" included many branching possibilities and error conditions that could arise, along with how to handle them. That is, the normal flow actually included alternative flows and exceptions. A better strategy is to pick a simple, default, well-behaved path through the use case and call that the normal flow. Then write separate alternative flows to cover variations on this and exception flows to describe the error conditions. This gives you a use case with multiple small packages of information, which is much easier to understand and manage than a huge use case that tries to handle every possibility in a single flow description.

Trap #4: Describing specific user interface elements and actions

Write "essential" use cases that describe the interactions between the user and the system at an abstract level, without incorporating user interface specifics. The use case description should not include a screen

design, although simple user interface prototypes can be valuable to facilitate the use case exploration. I don't even like to hear terminology in the use case that alludes to specific user interface controls. Saying "User clicks on OK" implies a GUI interface using a mouse and buttons. But what if it would make more sense to use a touch screen or speech recognition interface? Imposing premature design constraints in the use case can lead to a suboptimal design, unless you're adding a new capability to an existing application where the screens already exist.

Trap #5: Not using other requirement models

Analysts who begin employing use cases sometimes seem to forget everything else they know about requirements specification. Use cases are a great aid for exploring requirements for interactive systems, kiosks and Web sites. However, they do not work as well for event-driven real-time systems, data warehouses or batch processes.

Avoid the temptation to force fit all of your functional requirements into use cases. Supplement the use case descriptions with a detailed list of functional requirements, nonfunctional requirements, graphical analysis models, prototypes, a data dictionary and other representations of requirements information. Use cases are valuable in many situations, but add them to your analyst toolkit instead of replacing your current tools with them.

Planning requirements for multiple product releases

By Karl E. Wiegers, Ph.D. for SearchSoftwareQuality.com

Few software products come into their full, ultimate form with the initial release. Most products begin with an initial version that contains enough capabilities to make it useful. Then they evolve as new features are added and existing features are enriched through a series of releases.

As requirements are developed for such a product, the stakeholders must allocate requirements to forthcoming releases. They need to plan a release strategy that provides the maximum customer value consistent with budgets, schedules, resources and business objectives. This article describes two techniques for planning such release strategies.

Feature roadmaps

Many project teams, particularly those that are building commercial products, talk about product features. Many features can be implemented in multiple levels of detail or enrichment. You might begin with a rudimentary implementation of a particular feature and then add more capabilities to it over time. During release planning, you can determine which features will be implemented to what extent in each release. This results in a feature roadmap.

Here's an example. Suppose we've identified eight features for our product that we would ultimately like to implement to their full extent. We are planning four releases, adding more capabilities and enriching the various features from one release to the next. In analyzing these eight features, we've identified either four or five levels of increasing capability for each feature. A feature roadmap defines which levels of each feature we plan to implement in each release.

As an example, suppose our product is a computer security package and one feature is antivirus protection. In the first release we might begin with basic virus detection on incoming email. Feature level two for the antivirus feature might provide the ability to download virus signature updates automatically. The third feature level would let the user scan disk files and disinfect or quarantine any suspicious files found. Level four could provide the option of using advanced heuristics for detecting viruses and Trojans. Finally, the fifth feature level might enable automatic reporting of virus infections and suspicious files to the vendor.

You don't plan to implement all of these feature capabilities in the first release because that would delay your ability to get your product on the market in the desired time window. So you can plan which feature levels to allocate to each intended upcoming release.

Use case flows

Use case descriptions are made up of three major elements: the normal flow, zero or more alternative flows, and zero or more exception flows. The normal flow represents the default, most typical success scenario. Alternative flows describe other successful scenarios that involve variations from the normal flow, such as an optional pathway that the user selects. Exception flows address error conditions that have the

potential to prevent the use case from successful completion. Each exception is associated with a specific success flow, either the normal flow or an alternative flow.

During release planning, the stakeholders need to decide which portions of which use cases to implement in each forthcoming release. It's essential to prioritize the use cases for alignment with satisfying the project's business objectives. Within a use case, you can further prioritize the various success scenarios.

The normal flow is the starting point for implementing each use case. In addition, you must implement any exceptions associated with the normal flow. You can allocate alternative flows to the release that includes that use case's normal flow or to any other future planned releases. With each alternative flow that you implement, be sure to implement the associated exceptions. You might never implement certain alternative flows simply because their priority is too low.

Release planning involves reaching a balance between customer value, implementation cost and technical risk. Feature roadmaps and use case flow analysis are valuable techniques for planning releases by clustering related requirements into manageable packages.

About the author: *Karl E. Wiegers is Principal Consultant with Process Impact in Portland, Ore. He is also the author of More About Software Requirements: Thorny Issues and Practical Advice, Software Requirements, 2nd Edition; Peer Reviews in Software: A Practical Guide; and Creating a Software Engineering Culture.*

How to Detect Requirements Errors — A Guide to Slashing Software Costs and Shortening Development Time

By Adam Frankl and Tom King - March 2007

Requirements Errors

Requirements errors are the primary reason for project failure

According to the European Software Process Improvement Initiative survey of 3,800 software projects, fully 50% of all projects reported that requirements specifications were a “major problem,” making requirements specification the #1 source of software problems.

Developing precise and accurate requirements is a crucial step in the success of a development project or a new business process. However, organizations struggle with the consequences of poor communication about requirements. Communication challenges between business teams and technologists are chronic. Studies show that 60%-80% of project failures can be attributed directly to poor requirements. In the software development area within the United States alone, this represents a \$60 billion annual problem involving cancelled or failed projects.

Carnegie Mellon's SEI (Software Engineering Institute) reports that requirements problems are the single most frequent cause of failure in the following dimensions: projects that are significantly over budget, significantly past schedule, significantly reduced scope, significantly poor quality, not significantly used once delivered, or cancelled.

Requirements errors are the most common type of error

Industry data suggest that approximately 50% of product defects originate in the requirements. The SEI cites industry studies which report that the percentage of defects originating from requirements ranges from 42% to 64%.

Requirements errors constitute the majority of rework costs. Percentages as high as 70%, 80%, or even 85% of the rework effort on a development project can be traced to requirements defects. Since rework costs range from 30% to 50% of the typical total project budget, *requirements errors consume 25% to 40% of the total project budget.*

Requirements errors are the most expensive errors to fix

Compared to costs for changes that are discovered and made during the requirements gathering phase, cost increases resulting from late cycle changes can run from as little as 10 times to 100 times or greater.

Companies including GTE, TRW, IBM, and HP have measured and assigned costs to errors occurring at various phases of the project lifecycle. Although these studies were run independently, they all reached roughly the same conclusion: The cost to detect and repair an error during the requirements stage is between 5 to 10 times less than the cost to detect and repair an error during the coding stage. Furthermore, the cost to detect and repair an error during the maintenance stage is 100 times more. Altogether, there is as much as a 200:1 cost savings resulting from finding errors in the requirements stage versus finding errors in the maintenance stage of the software lifecycle.

Detecting requirements errors after deployment can be catastrophic

Requirements errors that are not detected until after deployment can be catastrophic. In safety critical systems, one study reports that “for the 34 safety incidents analyzed, 44% had inadequate specification as their primary cause.”

Clearly, *the early detection of requirements errors is essential to reduce software costs and development schedules*. In addition, there are intangible costs associated with a requirements error. Intangible costs include features that could have been delivered had the project's resources not been devoted to rework, loss of confidence on the part of customers, and accompanying lost and unrecoverable market share, revenue and profit. Taken together, these costs clearly demonstrate that a company cannot afford to ignore the benefits of better requirements development.

The Requirements Process

At the beginning of a software development project or a business process re-engineering project, it is common for business analysts to be assigned to get things started. An executive or business process owner will often provide a charter or overall business goal along with an initial budget. The business analysts then proceed through the four phases of requirements development:

- Elicitation – discovering requirements or processes
- Analysis – refining the requirements
- Specification – formalizing the requirements definition
- Validation – reviewing the formal requirements for accuracy and completeness

Elicitation

The business analysts interview process participants, subject matter experts, process owners (e.g., executive sponsors), and other stakeholders to elicit the business requirements. Requirements are typically gathered during cursory or formal meetings, where business information and functional needs are scribbled on a white board or typed into a Microsoft Word file or Excel spreadsheet by a business analyst.

Analysis

Requirements analysis involves refining the requirements to ensure that all stakeholders understand them and scrutinizing them for errors, omissions, and other deficiencies.

Specification

The business analysts then formalize the requirements by creating more formal documents. These documents may include Word documents put into formal requirements templates (e.g., in IBM's Rational RequisitePro). They may include converting the informal descriptions to flow diagrams using Microsoft Visio. They may include converting the requirements into a modeling language using one of several business process modeling tools. The most common standard for expressing a formal requirement includes the "use case," which is simply a description (a story) of users and systems interacting in a typical fashion to go through the various paths or parts of a business process or automated process.

Validation

An iterative approach demands a feedback loop so that review of requirements enables key business stakeholders to revisit initial requests. Once the business analysts have formalized the requirements, they can begin the validation process. They convene meetings to review the documents, flow charts, and use cases. They may also send the documents to various reviewers and solicit feedback.

Problems with the Requirements Process

Traditionally, these three phases have been accomplished through meetings and interviews, and have not benefited from automation or tools to make the job more effective. There are many issues that detract from the effectiveness of the requirements development phase of a project.

Impatience

Even though everyone involved in the project understands that the requirements phase is critical, they are impatient to get started on the “real work.” The artifacts produced by the requirements development phase do not represent very compelling evidence of real progress. Far more compelling are things like prototypes, screen shots, simulations and the like. So there is great pressure to shortcut the requirements phase and start to leverage all the other resources that are committed to the project (e.g., development resources, trainers, change managers, et al.).

Laborious Process

Unfortunately, the process of manually converting from informal to formal documents in the specification phase is laborious and time-consuming. Drawing Visio diagrams can take many hours of relatively non-creative work. Inputting requirements statements one-by-one into templates for business process modeling tools is similarly time-consuming and non-creative. Even inputting from informal Word documents into the Word template formats of requirements management systems is tedious. This not only tempts people to take shortcuts, it also drags out the timeframe so that the stakeholders, including subject matter experts, participants, developers, and process owners, lose context. For example, if a business analyst gathers input from some stakeholders and then tries to reconvene that group a week later to answer some questions about a Visio diagram, the business analyst will have trouble pulling the same people together and getting their minds all back into the process.

No Error Detection

Until recently, there were no tools to automatically detect and highlight requirements errors. This means that the formal documents have often just been formalized versions of unclear, inconsistent, and incomplete requirements. Reviewing formal documents by informal means (i.e., by asking reviewers to read the documents and try to point out errors) yields poor results. People are not computers, and the reviewers could use some computerized help to find errors and omissions.

Insufficient User Involvement

Customers often don't understand why it is so essential to work hard on gathering requirements and assuring their quality. It is often difficult to gain access to users and domain experts. Since requirements development is not the job of the users and subject matter experts, they may have difficulty understanding the technical expression of the specifications.

Symptoms of Poor Requirements

What are the implications of an out of control requirements process? The first symptom is a requirements gathering process that drags on for months, adding a huge cost in time to the implementation process. But the main business pain is that the development team does not know exactly what they should be developing. There are several pain points that this situation presents:

- The development time becomes much too long and high cost, as the developers are forced to go back to the business leaders again and again to redefine unclear requirements.

- The development project becomes difficult to manage because it cannot be correctly phased. It becomes impossible to identify complete scenarios that can be implemented first and then expanded — raising risk by pushing bad news to near deadlines.
- The quality of the application itself becomes poor, as it does not match the real business requirements.

What does all this mean to the bottom line? Longer development times mean that the ROI of the entire application is delayed, and thus reduced. But even more important than the reduction in the ROI is that a poor quality application means that the desired business outcomes are not achieved.

Applying Automation to Detect Requirements Errors

Best Practices

Errors in requirements gathering can be minimized, but not eliminated. Proper validation is essential in catching errors early when they are inexpensive to fix. A number of best practices have been identified for requirements development. Among these are use cases as the best practice for developing functional specifications of requirements, and visually modeling software requirements. However, these best practices do not address the fundamental issue in requirements development: subject matter experts and sponsoring executives have been proven to be ineffective in reviewing 500 page textual requirements encyclopedias and convoluted multi-model program diagrams in order to validate them.

Automation can help

RAVEN automatically parses use cases and creates visual models from plain business English text. RAVEN is an acronym for Requirements Authoring and Validation Environment. These management review diagrams are designed to be read and reviewed by business leaders, yet at the same time are precise enough for implementation. Analysts gain immediate visual feedback on their use cases. Executives and subject matter experts can quickly validate visual models, and unambiguous and precise descriptions of functional requirements can be handed off to technologists.

RAVEN transforms requirements development by making errors visually obvious and thus dramatically compressing the timeframe for completing this work. This allows the business analyst to see errors and to transform the unstructured English into “requirements English” that specifies the use case clearly, consistently, and completely.

Online Collaboration

Today’s validation processes add little value to the requirements process. Either a 500 page unread textual requirements document is emailed around, or hours-long review meetings put executive stakeholders to sleep. Communicating complex information precisely is difficult, especially when the various stakeholders and users are distributed geographically and have different points of view and ways to communicate.

Today’s Web 2.0 technologies enable communities to coalesce and interact around the basis of shared interests. Interactive online communities can overcome the barriers of geographic distribution and diverse points of view. RAVEN includes capabilities enabling online collaborative review of requirements. Any stakeholder with a web browser can access the latest specifications and provide their own comments. Rapid feedback and continuous validation enables requirements errors to be quickly detected and eliminated.

The end result is a shorter requirements development phase that produces a more accurate business requirements specification. This leads to a shorter overall development time and a higher quality application as more business requirements are met.